Communication micro-services via kafka
2026

1 Objectifs

Dans le TP1, les microservices communiquaient directement au moyen d'une API REST. Dans ce TP, nous allons introduire une communication **asynchrone** avec Apache Kafka. À la fin du TP, vous serez capables de :

- comprendre le principe de la communication par événements ;
- utiliser Kafka avec des microservices spring boot ;
- produire un événement dans un topic Kafka ;
- consommer un événement depuis un topic Kafka ;
- faire communiquer plusieurs microservices sans appel REST direct ;
- exécuter les microservices et Kafka avec Docker.

1.1 Architecture du TP

- Dans le TP1, la communication entre micro-services était directe.
- Dans ce TP, les microservices communiquent par l'intermédiaire de Kafka.
- Le microservice `commandes` produit un événement.
- Le microservice `clients` consomme cet événement.

Proposez une architecture globale de ce système.

1.2 Principe d'un événement

- Lorsqu'une nouvelle commande est créée, le microservice `commandes` publie un événement dans Kafka.
- Par exemple :
  1. ```
     {
     "commande_id": 10,
     "client_id": 2,
     "produit": "PC",
     "montant": 1200
     }
     ```
  2. Cet événement est envoyé dans le topic :

     ```
     commandes-creees
     ```

  3. Le microservice `clients` consomme ensuite cet événement.
     ```

1.3 Installation de Kafka avec Docker

— Pour ce TP, Kafka sera exécuté dans un conteneur Docker. Créer un fichier *docker-compose.yml* avec l'infrastructure Kafka.

— Donner l'architecture globale du système.

— Démarrer l'infrastructure avec :

```
docker compose up -d
```

— Vérifier les conteneurs :

```
docker ps
```

1.4 Création du producteur commandes Kafka

Le microservice commandes doit publier sur un topic Kafka un message JSON contenant les informations d'une commande lorsqu'une nouvelle commande est reçue. Précisez et implémentez les dépendance(s) nécessaire(s) :

— la connexion du producteur au broker Kafka ;

— le topic Kafka utilisé ;

— la structure du message JSON envoyé ;

— l'envoi du message depuis l'API REST du microservice Commandes.

— publication d'une commande.

1.5 Création du consommateur clients kafka

— Modifier le microservice `clients` afin qu'il consomme les événements `commandes-creees` produits par le microservice `commandes`. À chaque commande reçue, afficher le client, le produit et le montant, puis maintenir et afficher le nombre total de commandes effectuées par chaque client.

— Observer la communication entre les différentes entités de cette architecture, notamment les échanges de messages via Kafka entre les microservices.

1.6 Arrêter un microservice

1. Arrêter le microservice `clients`.
2. Créer plusieurs nouvelles commandes.
3. Puis redémarrer le microservice `clients`.
4. Observer ce qui se passe :
   — Les événements produits pendant l'arrêt du microservice `clients` ont-ils été perdus ?
   — Observer les messages consommés par `clients`.

1.7 Ajouter un deuxième consommateur

Créer un troisième microservice qui consomme également le topic *commandes-creees*. Il doit calculer :

— le nombre total de commandes pour un produit donné ;

— le chiffre d'affaires total pour un mois donné (apporter les changements aux évènements déposés si nécessaire) ;

8. Kafka et groupes de consommateurs

1. Créer deux consommateurs appartenant au même groupe : group_id = "groupe-A"
2. Observer la distribution des messages.
3. Puis créer deux consommateurs appartenant à des groupes différents : group_id = "clients-group" et group_id = "statistiques-group"
4. Quelle différence observez-vous entre les deux configurations ?